

Numerical Methods 1

Lecture 3: Numerical Integration

1. 基础概念

Introduction to Numerical Integration 数值积分简介

Quadrature 求积法 is the term used for numerical evaluation of a *definite* integral 定积分, i.e., computing the area under a curve over a specific interval $[a, b]$. When dealing with complex functions or discrete data points, analytical integration may be impossible or impractical, making numerical methods essential. Quadrature is the term used for numerical evaluation of a *definite* (i.e. over a range $[a, b]$) integral, or in 1D finding the area under a curve.

If we have a function $f(x)$ defined between a and b , the *definite integral* 定积分 is:

$$I := \int_a^b f(x) dx$$

The fundamental principle underlying all quadrature methods is the **additivity property** 可加性 of definite integrals:

$$\int_a^b f(x) dx = \int_a^c f(x) dx + \int_c^b f(x) dx$$

This allows us to divide the integration domain into smaller subintervals and approximate the integral over each subinterval using simple geometric shapes.

2. Overview of Quadrature Methods 求积方法概述

The accuracy and computational cost of quadrature methods depend on:

The **approximation method** 近似方法 used within each subinterval. The **number of subintervals** 子区间数量 (smaller intervals generally yield higher accuracy) The **smoothness** 光滑性 of the integrand

We will examine five fundamental quadrature rules with increasing levels of sophistication and accuracy.

3. Midpoint Rule 中点法则

The **midpoint rule** 中点法则 is the simplest quadrature method, also known as the **rectangle method** 矩形法.

3.1. Formula 公式

For a subinterval $[x_i, x_{i+1}]$, the midpoint rule approximates the integral by:

$$I_M^{(i)} = (x_{i+1} - x_i) \times f\left(\frac{x_{i+1} + x_i}{2}\right)$$

The total integral is:

$$I_M = \sum_{i=0}^{n-1} f\left(\frac{x_{i+1} + x_i}{2}\right) (x_{i+1} - x_i)$$

4. Trapezoidal Rule 梯形法则

The **trapezoidal rule** 梯形法则 uses trapezoids instead of rectangles, with the top edge being a linear interpolation between the function values at the interval endpoints.

4.1. Formula 公式

For a subinterval $[x_i, x_{i+1}]$:

$$I_T^{(i)} = (x_{i+1} - x_i) \times \frac{f(x_{i+1}) + f(x_i)}{2}$$

The total integral is:

$$I_T = \sum_{i=0}^{n-1} \frac{f(x_{i+1}) + f(x_i)}{2} (x_{i+1} - x_i)$$

5. Simpson's Rule 辛普森法则

Simpson's rule 辛普森法则 is derived by combining the midpoint and trapezoidal rules to cancel their leading-order errors, resulting in a higher-order method.

5.1. Formula 公式

For an interval $[a, b]$ with midpoint $c = (a + b)/2$:

$$I_S = \frac{b - a}{6} [f(a) + 4f(c) + f(b)]$$

6. Composite Simpson's Rule 复合辛普森法则

The **composite Simpson's rule** 复合辛普森法则 is an optimized version that eliminates redundant function evaluations by using only the given grid points.

6.1. Formula 公式

For n intervals (where n must be even):

$$I_{CS} = \frac{\Delta x}{3} \left[f(x_0) + 2 \sum_{i=1}^{n/2-1} f(x_{2i}) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + f(x_n) \right]$$

7. Weddle's Rule 韦德尔法则

Weddle's rule 韦德尔法则, also known as **extrapolated Simpson's rule** 外推辛普森法则, uses **Richardson extrapolation** 理查森外推 to achieve even higher accuracy.

7.1. Formula 公式

Using two Simpson's rule estimates with n and $2n$ intervals:

I_S : Simpson's rule with n intervals

I_{S2} : Simpson's rule with $2n$ intervals

Since Simpson's rule has $O(h^4)$ accuracy, I_{S2} should be approximately $2^4 = 16$ times more accurate than I_S .

The relationship is:

$$I_W - I_S = 16(I_W - I_{S2})$$

Solving for I_W :

$$I_W = I_{S2} + \frac{I_{S2} - I_S}{15}$$

8. Comparative Error Analysis 比较误差分析

8.1. Convergence Rates 收敛速率

The convergence rates in terms of interval size h are:

Method 方法	Order 阶数	Error Scaling 误差标度
Midpoint 中点法	$O(h^2)$	Error $\propto h^2$
Trapezoidal 梯形法	$O(h^2)$	Error $\propto h^2$
Simpson's 辛普森法	$O(h^4)$	Error $\propto h^4$
Composite Simpson's 复合辛普森法	$O(h^4)$	Error $\propto h^4$
Weddle's 韦德尔法	$O(h^6)$	Error $\propto h^6$

8.2. Practical Considerations 实际考量

Function Evaluation Cost 函数计算成本: Higher-order methods require more function evaluations per interval but achieve better accuracy

Numerical Stability 数值稳定性: Very high-order methods like Weddle's rule may suffer from numerical instability when errors approach machine precision

Smoothness Requirements 光滑性要求: Higher-order methods assume the integrand is sufficiently smooth; for non-smooth functions, lower-order methods may be more robust

Implementation Complexity 实现复杂度: Composite forms are more efficient but require careful implementation to avoid redundant calculations

9. Code

一个常见的问题是针对某个函数执行算法，然后计算不同步长下的errors，画图。第一步还是要输入一些数据和参数。这里主要关注true_int这个数据，可以理解成真实的结果，有的时候题目会给你一个具体的数字，有的时候会让你算，这个要自己看明白再替换，然后函数和范围也要自己替换

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
import scipy.interpolate as si
import scipy.integrate as integrate
import scipy.linalg as sl
import scipy.optimize as sop
try:
    from scipy.integrate import simps
except ImportError:
    from scipy.integrate import simpson as simps

'''设定积分区间的左右边界'''
# Integration interval and test function
# Set the boundaries of the integral range
xl = -2
xr = 2

'''定义一个复杂的测试函数：包含三角函数、指数函数和常数项'''
# Test integrand:
#  $f(x) = x \sin(\pi x) - \exp(-x) + 10$ 
# Includes a trigonometric term, an exponential term, and a constant offset
def f(x):
    return x*np.sin(np.pi*x) - np.exp(-x) + 10

'''使用辛普森法则，通过不断增加数据点数来计算真实解，这里使用SciPy的simpson函数处理'''
# Reference ("true") integral:
# Use SciPy Simpson on a very fine uniform grid as a high-accuracy benchmark
# This is still numerical, but typically very accurate for smooth functions
for interval_sizes in [2**i for i in range(17)]: # 1, 2, 4, ..., 65536 subintervals
    xx = np.linspace(xl, xr, interval_sizes + 1) # n subintervals => n+1 points
    print(interval_sizes, simps(f(xx), xx) )

'''将最后一次计算（最高精度）的结果作为"真实"积分值，用于后续误差分析的参考标准'''
# use the finest-grid result as reference
true_int = simps(f(xx), xx)
```

```

# -----

# 求积规则
# Quadrature rules

'''中点法则的实现：使用矩形面积来近似积分，每个矩形的高度取自区间中点的函数值，计算每个
# Composite midpoint rule:
# Split [a,b] into N equal subintervals, evaluate the function at each mi
# Returns integral approximation of function(x) over [a,b].
def midpoint_rule(a, b, function, number_intervals=10):

    interval_size = (b - a)/number_intervals

    assert interval_size > 0
    assert type(number_intervals) == int

    '''初始化积分结果为零'''
    # Initialise to zero the variable that will contain the cumulative su
    I_M = 0.0

    '''找到第一个矩形的中点坐标'''
    # Find the first midpoint -- i.e. the centre point of the base of the
    mid = a + (interval_size/2.0)

    '''循环直到超过右端点b，计算并累加每个矩形的面积'''
    # and loop until we get past b, creating and summing the area of each
    while (mid < b):

        '''计算当前矩形面积：宽度 × 中点函数值'''
        # Find the area of the current rectangle and add it to the runnin
        # this involves an evaluation of the function at the subinterval
        I_M += interval_size * function(mid)

        '''移动到下一个区间的中点'''
        # Move the midpoint up to the next centre of the interval
        mid += interval_size

    '''返回累计的积分结果'''
    # Return our running total result
    return I_M

'''非复合版本的梯形法则的实现'''
# Composite trapezoidal rule:
# For each subinterval, compute the trapezoid area using both endpoints.
def trapezoidal_rule(a, b, function, number_intervals=10):

    '''计算每个小区间的宽度'''
    # Calculate the width of each interval
    interval_size = (b - a)/number_intervals

    assert interval_size > 0
    assert type(number_intervals) == int

    '''初始化积分结果为零'''
    # Initialise to zero the variable that will contain the cumulative su
    I_T = 0.0

    '''循环创建每个梯形并计算面积'''
    # Loop to create each trapezoid

```

```

    for i in range(number_intervals):

        '''计算当前梯形的面积并累加到总积分中，梯形面积公式：宽度 × (左高度 + 右高度)
        # Find the area of the current trapezoid and add it to the running total
        x0 = a + i * interval_size
        x1 = x0 + interval_size
        I_T += interval_size * (function(x0) + function(x1)) / 2.0

        '''返回累计的积分结果'''
    # Return our running total result
    return I_T

'''辛普森法则的实现：使用抛物线来近似被积函数，这个版本在每个子区间的端点和中点都计算函数值
# Simpson's rule applied per-subinterval:
# On each subinterval [x0,x1], use midpoint xm and apply: (h/6) * [f(x0) + 4*f(xm) + f(x1)]
def simpsons_rule(a, b, function, number_intervals=10):

    '''计算每个小区间的宽度'''
    # Calculate the width of each interval
    interval_size = (b - a) / number_intervals

    assert interval_size > 0
    assert type(number_intervals) == int

    '''初始化积分结果为零'''
    # Initialise to zero the variable that will contain the cumulative sum
    I_S = 0.0

    '''循环对每个区间应用辛普森公式，其中h是区间宽度，c是中点'''
    # Loop to evaluate Simpson's formula over each interval. Calculate the sum
    for i in range(number_intervals):

        '''计算当前区间的左端点、中点和右端点'''
        # Find a, c, and b
        x0 = a + i * interval_size
        xm = x0 + 0.5 * interval_size
        x1 = x0 + interval_size
        I_S += (interval_size / 6.0) * (function(x0) + 4.0 * function(xm) + function(x1))

    '''返回累计的积分结果'''
    # Return our running total result
    return I_S

'''复合辛普森法则的实现'''
# Standard composite Simpson's rule using (N+1) grid points (efficient):
# Requires N to be even.
# Formula:
# I ≈ (h/3) * [ f(x0) + f(xN) + 4 * sum_{odd i} f(xi) + 2 * sum_{even i} f(xi) ]
def simpsons_composite_rule(a, b, function, number_intervals=10):

    assert number_intervals % 2 == 0, "number_intervals is not even"

    '''计算每个小区间的宽度'''
    # Calculate the width of each interval
    interval_size = (b - a) / number_intervals

    '''从两个端点的函数值开始'''
    # start with the two end member values
    I_cS2 = function(a) + function(b)

```

```

'''添加系数为4的项（对应奇数索引点），这些点是每个子区间对的中点'''
# add in those terms with a coefficient of 4
for i in range(1, number_intervals, 2):
    I_cS2 += 4 * function(a + i * interval_size)

'''添加系数为2的项（对应偶数索引点，除了端点），这些点是相邻子区间对之间的分界点'''
# and those terms with a coefficient of 2
for i in range(2, number_intervals-1, 2):
    I_cS2 += 2 * function(a + i * interval_size)

return I_cS2 * (interval_size / 3.0)

'''韦德尔法则的实现，这是一种基于辛普森法则的外推法，用于提高精度，通过比较不同步长的辛
# "Weddle" step implemented here via Richardson extrapolation on Simpson:
# Compute Simpson with N intervals (S) and with 2N intervals (S2), then c
# This uses the fact that Simpson's leading error term scales like h^4,
# so the extrapolation cancels the dominant error term.
def weddles_rule(a, b, function, number_intervals=10):

    '''使用给定区间数计算辛普森积分'''
    # Calculate Simpson's integral using the given interval numbers
    S = simpsons_composite_rule(a, b, function, number_intervals)

    '''使用两倍区间数（更精细划分）计算辛普森积分'''
    # Calculate Simpson's integral using a double interval number
    S2 = simpsons_composite_rule(a, b, function, number_intervals*2)

    '''韦德尔外推公式：S2 + (S2 - S)/15'''
    '''这里利用了辛普森法则的误差阶数来进行外推修正'''
    return S2 + (S2 - S)/15.

# -----

'''定义不同数值积分方法的区间划分数'''
'''使用2的幂次方序列：1, 2, 4, 8, 16, 32, 64, 128, 256, 512'''
# Run methods for different numbers of intervals and compute errors
interval_sizes_M = [2**i for i in range(10)]
interval_sizes_T = [2**i for i in range(10)]
interval_sizes_S = [2**i for i in range(10)]
interval_sizes_W = [2**i for i in range(10)]

'''韦德尔法则需要偶数个区间，所以去掉第一个元素(1)，最终序列变为：2, 4, 8, 16, 32,
# due to need for even number of intervals lose the first
interval_sizes_W = interval_sizes_W[1:]

'''初始化误差数组，用于存储每种方法在不同区间数下的误差'''
# Initialize the error array to store the errors of each method under dif
errors_M = np.zeros_like(interval_sizes_M, dtype='float64')
errors_T = np.zeros_like(interval_sizes_T, dtype='float64')
errors_S = np.zeros_like(interval_sizes_S, dtype='float64')
errors_W = np.zeros_like(interval_sizes_W, dtype='float64')

'''初始化面积数组，用于存储每种方法计算得到的积分值'''
# Initialize the area array to store the integral values calculated by ea
areas_M = np.zeros_like(interval_sizes_M, dtype='float64')
areas_T = np.zeros_like(interval_sizes_T, dtype='float64')
areas_S = np.zeros_like(interval_sizes_S, dtype='float64')
areas_W = np.zeros_like(interval_sizes_W, dtype='float64')

'''计算中点法则在不同区间数下的积分值和误差'''

```

```

# Calculating the integral values and errors in for Midpoint
for (i, number_intervals) in enumerate(interval_sizes_M):
    areas_M[i] = midpoint_rule(xl, xr, f, number_intervals)
    errors_M[i] = abs(areas_M[i] - true_int)

'''计算梯形法则在不同区间数下的积分值和误差'''
# Calculating the integral values and errors in for Trapezoidal
for (i, number_intervals) in enumerate(interval_sizes_T):
    areas_T[i] = trapezoidal_rule(xl, xr, f, number_intervals)
    errors_T[i] = abs(areas_T[i] - true_int)

'''计算辛普森法则在不同区间数下的积分值和误差'''
# Calculating the integral values and errors in for Simpson
for (i, number_intervals) in enumerate(interval_sizes_S):
    areas_S[i] = simpsons_rule(xl, xr, f, number_intervals)
    errors_S[i] = abs(areas_S[i] - true_int)

'''计算韦德尔法则在不同区间数下的积分值和误差'''
# Calculating the integral values and errors in for Weddle
for (i, number_intervals) in enumerate(interval_sizes_W):
    areas_W[i] = weddles_rule(xl, xr, f, number_intervals)
    errors_W[i] = abs(areas_W[i] - true_int)

# -----

'''绘制不同数值积分方法的收敛性比较图'''
# Plot convergence (log-log plot of error vs number of intervals)
fig = plt.figure(figsize=(7, 5))
ax1 = plt.subplot(111)

ax1.loglog(interval_sizes_S, errors_S, 'ro-', lw=2, label='Simpson')
ax1.loglog(interval_sizes_T, errors_T, 'bo-', lw=2, label='Trapezoidal')
ax1.loglog(interval_sizes_M, errors_M, 'ko-', lw=2, label='Midpoint')
ax1.loglog(interval_sizes_W, errors_W, 'go-', lw=2, label='Weddles')

ax1.set_xlabel('log(number_intervals)', fontsize=16)
ax1.set_ylabel('log(error)', fontsize=16)
ax1.set_title('Quadrature rule convergence', fontsize=16)

ax1.legend(loc='best', fontsize=14)

plt.show()

```